

Arquitectura de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 7:

Administración, Seguridad y
Escalabilidad de Bases de Datos



Arquitectura de Datos

Ya sabemos dónde y cómo almacenar los datos

Hasta ahora aprendimos a:

- modelar información;
- implementar y consultar bases relacionales;
- optimizar estructuras y consultas;
- utilizar modelos NoSQL;
- almacenar series temporales y datos geoespaciales;
- utilizar caché y almacenamiento de objetos;
- realizar búsqueda vectorial;
- incorporar RAG y Text-to-SQL;
- combinar mecanismos de persistencia.

Ahora aparece otra pregunta:

¿cómo administramos y protegemos esa arquitectura cuando comienza a utilizarse realmente?



Elegir dónde almacenar los datos no termina el problema.
También necesitamos controlar cómo se accede y cómo se opera sobre ellos.

Cambio de perspectiva

Hasta ahora organizamos los datos, hoy empezamos a administrar su operación



Una arquitectura de datos también debe definir quién puede utilizar la información y bajo qué condiciones.

Objetivos de la clase

Administrar una base que ya forma parte de un sistema real

Vamos a aprender a:

- administrar usuarios, roles y permisos;
- aplicar mínimo privilegio;
- comprender arquitecturas multi-tenant;
- aislar información mediante ``tenant_id``;
- implementar Row Level Security;
- aplicar aislamiento también a RAG y Text-to-SQL;
- comprender concurrencia y bloqueos;
- realizar backup y restore;
- monitorear la base;
- comprender replicación;
- particionar información;
- diferenciar escalabilidad vertical y horizontal.



Una arquitectura debe funcionar, pero también debe poder operarse de manera segura, recuperable y controlada.

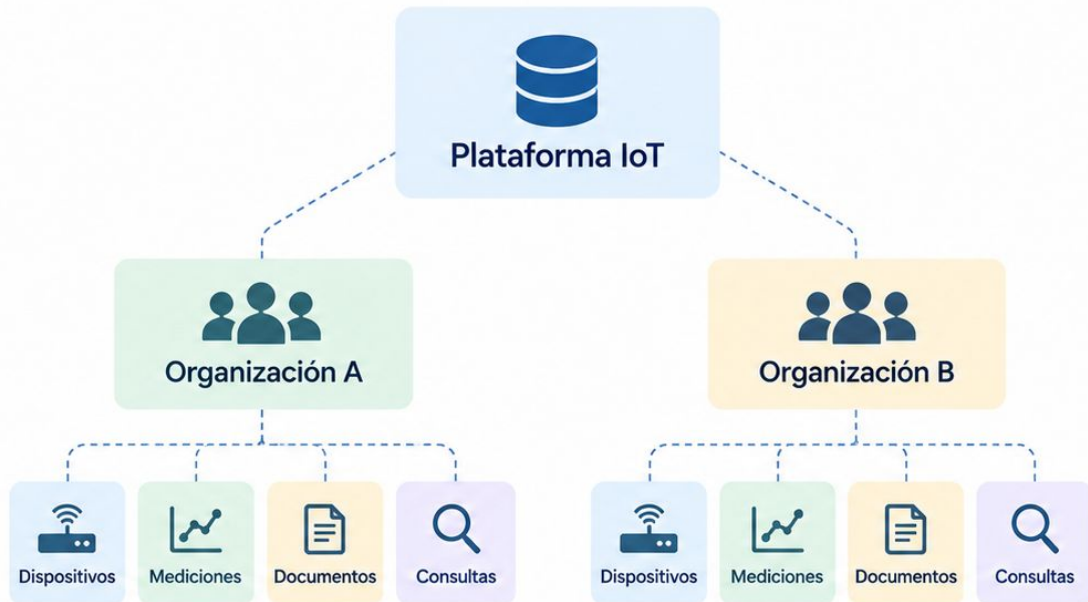
El caso vuelve a evolucionar

Varias organizaciones utilizan la misma plataforma

Ahora el sistema posee:

- diferentes organizaciones;
- usuarios de cada organización;
- miles de dispositivos;
- mediciones históricas;
- documentos técnicos;
- consultas mediante lenguaje natural;
- procesos automáticos;
- múltiples accesos simultáneos.

Una organización no debería poder acceder a los datos de otra.



Compartir una plataforma no significa compartir la información.

Arquitectura de Datos

Agenda del día

- Usuarios, autenticación y roles.
- Permisos y mínimo privilegio.
- Aplicaciones multi-tenant.
- Estrategias de aislamiento.
- tenant_id.
- Row Level Security.
- Seguridad aplicada a RAG y Text-to-SQL.
- Transacciones y concurrencia.
- Aislamiento y bloqueos.
- Backup y recuperación.
- Monitoreo y mantenimiento.
- Replicación.
- Particionamiento.
- Escalabilidad vertical y horizontal.
- Práctica integradora.



Vamos a recorrer desde el acceso de un usuario hasta el crecimiento de toda la infraestructura.

1

Usuarios, Roles y Permisos



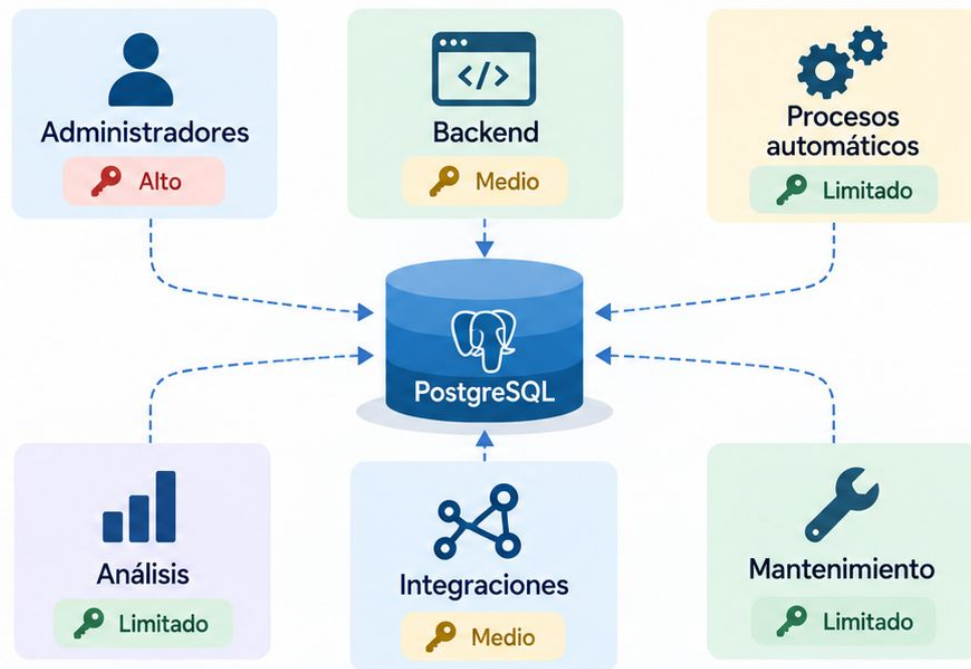
La base recibe conexiones

No solamente existen usuarios humanos

Una base puede recibir conexiones desde:

- administradores;
- aplicaciones backend;
- procesos automáticos;
- herramientas de análisis;
- servicios de integración;
- tareas de mantenimiento.

Cada uno necesita capacidades diferentes.



No todas las conexiones deberían tener los mismos privilegios.

Dos preguntas diferentes

¿Quién sos? y ¿qué podés hacer?



Autenticación

¿Quién intenta conectarse?



Autorización

¿Qué operaciones tiene permitidas?

Autenticación



Verificamos
quién sos

Autorización



Verificamos
qué podés hacer

Datos



Acceso a
la información

Un usuario puede autenticarse correctamente y aun así no tener permiso para modificar determinada información.



Poder conectarse no significa poder hacer cualquier operación.

No usar siempre el mismo usuario

Administrar y operar son funciones diferentes



Administrador

Puede necesitar:

- crear usuarios;
- modificar estructuras;
- administrar permisos;
- realizar mantenimiento.



Aplicación

Puede necesitar solamente:

- consultar dispositivos;
- insertar mediciones;
- modificar determinadas configuraciones.

Administrador



Acceso amplio

- ✓ Gestionar usuarios
- ✓ Modificar estructuras
- ✓ Administrar permisos
- ✓ Realizar mantenimiento

Aplicación



Acceso limitado

- ✓ Consultar dispositivos
- ✓ Insertar mediciones
- ✓ Modificar configuraciones
- ✗ Sin permisos administrativos



La aplicación no debería conectarse utilizando una cuenta administrativa.

Roles

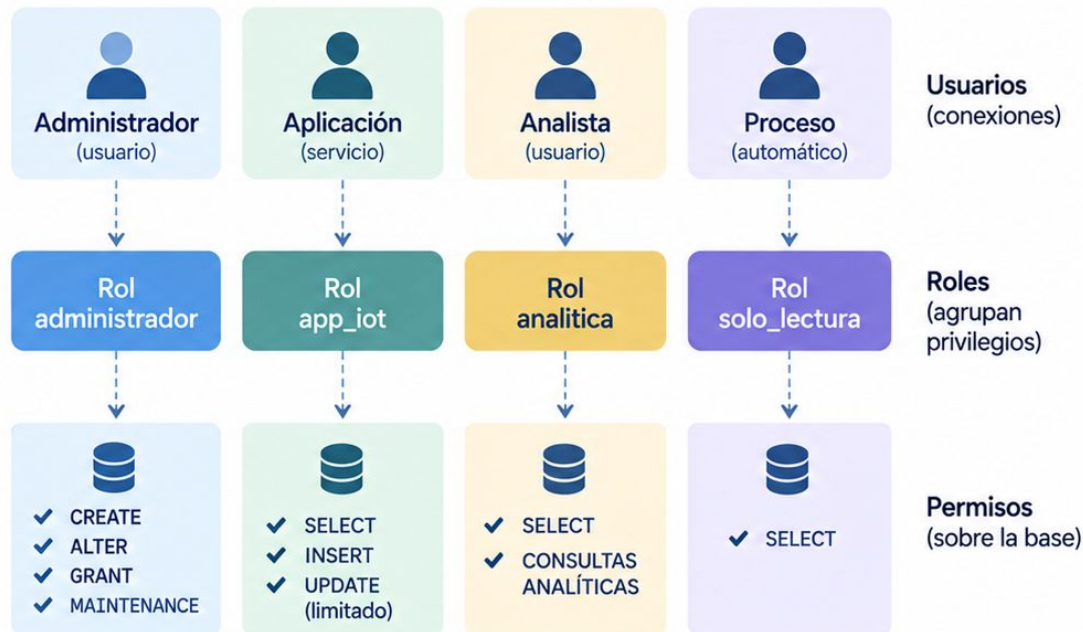
Agrupar privilegios según una función

```
CREATE ROLE app_iot LOGIN;
```

Podemos pensar roles como:

- administrador
- aplicacion
- solo_lectura
- analitica

Los permisos se asignan según las responsabilidades de cada uno.



Un rol representa qué necesita hacer una conexión dentro del sistema.

Permisos

Definir qué operaciones están autorizadas

```
GRANT SELECT  
ON dispositivos  
TO app_iot;
```

```
GRANT INSERT  
ON mediciones  
TO app_iot;
```

También podemos retirar permisos:

```
REVOKE DELETE  
ON mediciones  
FROM app_iot;
```



Permisos del rol app_iot

Tabla	SELECT	INSERT	UPDATE	DELETE
dispositivos	✓	—	—	—
mediciones	✓	✓	—	✗
ubicaciones	—	—	—	—
organizaciones	—	—	—	—



Los permisos deberían habilitar solamente las operaciones necesarias.

Mínimo privilegio

Solamente lo necesario para cumplir una función

Supongamos que el backend necesita:

- leer dispositivos;
- consultar ubicaciones;
- insertar mediciones.

No necesita:

- ✗ crear tablas;
- ✗ eliminar bases;
- ✗ crear usuarios;
- ✗ administrar permisos.



Cada componente debería disponer del menor conjunto de privilegios necesario.

Otros motores

Cambian los mecanismos, no la necesidad

	PostgreSQL	roles + GRANT / REVOKE
	MySQL	usuarios + roles + privilegios
	SQL Server	logins + users + roles + permisos
	MongoDB	usuarios + roles
	Redis	ACL
	Cassandra / ScyllaDB	roles + permisos



Toda plataforma necesita alguna forma de determinar quién puede realizar cada operación.

2

Multi-Tenant y Aislamiento de Datos



Tenant

Una organización dentro de una plataforma compartida

Una aplicación multi-tenant puede atender:

- Organización A
- Organización B
- Organización C
- ...

Compartiendo parte de la infraestructura.
Cada tenant posee información propia.



Compartir infraestructura no implica compartir datos.

Aislamiento de los datos

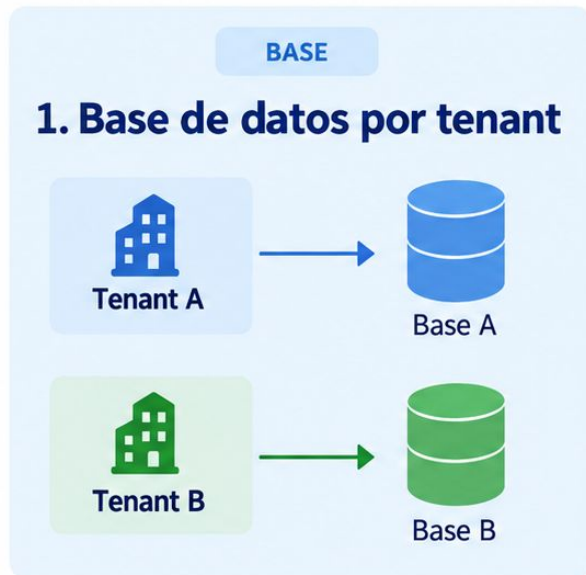
Una organización no debería observar información de otra



El aislamiento forma parte del diseño de la arquitectura de datos.

Estrategias de aislamiento

¿Cómo aislamos los datos de cada tenant?



En los tres casos buscamos que la información de una organización permanezca separada de la correspondiente a las demás.



Las estrategias separan los datos en niveles diferentes: base, esquema o filas.

Distintos niveles de separación

Cada alternativa modifica la arquitectura

Estrategia	Separación	Característica
Base por tenant	Base de datos	Cada tenant posee su propia base
Esquema por tenant	Esquema	Comparten la base, pero no las estructuras
Tablas compartidas	Filas	Comparten tablas y cada fila identifica al tenant

La elección puede depender de:

- cantidad de tenants;
- nivel de aislamiento requerido;
- complejidad operativa;
- mantenimiento;
- evolución del sistema.



No existe una única estrategia multi-tenant adecuada para todos los escenarios.

Una base por tenant

Separar completamente los datos

Cada organización utiliza una base independiente.



Esto puede facilitar:

- separación de datos;
- políticas particulares;
- operaciones independientes.



Pero también aumenta:

- cantidad de bases;
- conexiones;
- backups;
- migraciones;
- tareas de administración.



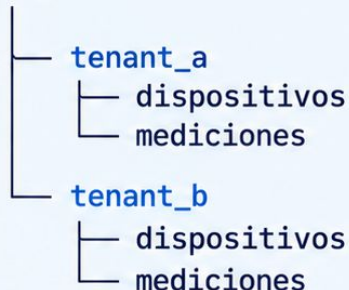
Mayor separación también puede significar mayor complejidad operativa.

Un esquema por tenant

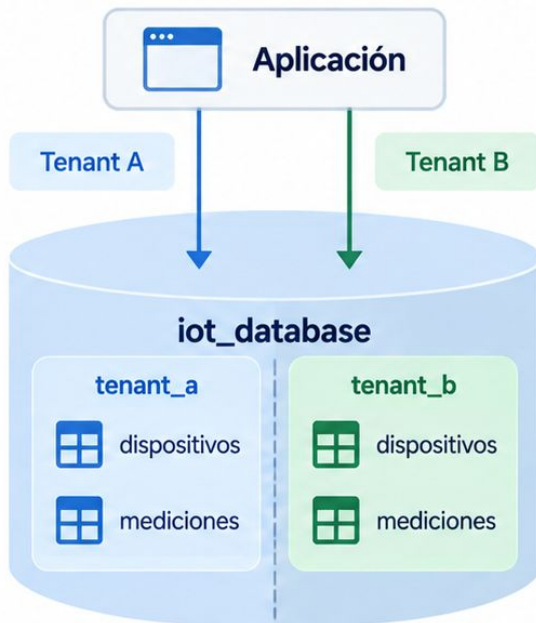
Compartir la base, separar las estructuras

Una misma base puede contener:

`iot_database`



Los tenants comparten el mismo servidor y la misma base, pero utilizan estructuras lógicamente separadas.



Ventajas

- ✓ Estructuras separadas
- ✓ Uso de una sola base
- ✓ Administración centralizada
- ✓ Facilita el aislamiento lógico

Consideraciones

- ✗ Mayor complejidad en el diseño
- ✗ Migraciones por esquema
- ✗ Gestión de permisos por tenant
- ✗ Puede crecer el número de esquemas



El esquema agrega una frontera lógica dentro de una misma base de datos.

Tablas compartidas

Identificar a quién pertenece cada fila

En lugar de duplicar estructuras, todos los tenants pueden utilizar las mismas tablas.

dispositivos

dispositivo_id
tenant_id
nombre
numero_serie

En nuestro caso, `organizacion_id` puede cumplir conceptualmente el rol de `tenant_id`.

dispositivos				
	dispositivo_id	tenant_id	nombre	numero_serie
 Organización A	1	tenant A	Ambiente A01	SN-A01
 Organización A	2	tenant A	Ambiente A02	SN-A02
 Organización B	3	tenant B	Planta B01	SN-B01



Cuando compartimos tablas, cada fila necesita indicar claramente a qué tenant pertenece.

El filtro parece sencillo

Hasta que alguna consulta lo olvida

Supongamos que el usuario pertenece a: `tenant_id = 1`



CON FILTRO

La consulta incluye el tenant del usuario

```
SELECT *  
FROM dispositivos  
WHERE tenant_id = 1;
```

Tenant A

id	nombre del dispositivo
A01	Impresora Oficina
A02	Sensor Lobby

Recordado



vs.

Olvidado



SIN FILTRO

La consulta olvida el tenant del usuario

```
SELECT *  
FROM dispositivos;
```

Tenant A

id	nombre del dispositivo
A01	Impresora Oficina
A02	Sensor Lobby

Tenant B

id	nombre del dispositivo
B01	Cámara Entrada
B02	Router Piso 2



También aparecen
datos de otro tenant



El aislamiento no debería depender solamente de recordar un `WHERE tenant_id = ...` en cada consulta.

Row Level Security

Aplicar el aislamiento desde la base

PostgreSQL permite definir políticas que controlan qué filas puede utilizar una determinada sesión.

RLS puede intervenir sobre operaciones como:

● **SELECT** ● **INSERT** ● **UPDATE** ● **DELETE**

En nuestro escenario podemos utilizarlo sobre tablas compartidas identificadas mediante

`tenant_id` .

Tablas compartidas

Una misma tabla contiene datos de varios tenants

id	tenant_id	nombre del dispositivo	ubicación
1	1	Impresora Oficina	Piso 1
2	1	Sensor Lobby	Piso 1
3	2	Cámara Entrada	Piso 2
4	2	Router Piso 2	Piso 2

tenant_id

Criterio de identificación del tenant actual

`tenant_id = 1`



Política RLS

Filtra las filas según las políticas definidas para la sesión actual.

Aplica sobre:

- **SELECT**
- **INSERT**
- **UPDATE**
- **DELETE**

Filas autorizadas

Solo se devuelven las filas del tenant actual

id	tenant_id	nombre del dispositivo	ubicación
1	1	Impresora Oficina	Piso 1
2	1	Sensor Lobby	Piso 1



RLS no es otra estrategia de multi-tenancy: es un mecanismo para hacer cumplir políticas sobre las filas de una tabla compartida.

Habilitar RLS

Preparar una tabla para aplicar políticas

En PostgreSQL:

```
ALTER TABLE dispositivos  
ENABLE ROW LEVEL SECURITY;
```

Podemos aplicar el mismo principio a otras estructuras que contienen información perteneciente a organizaciones:

- dispositivos
- mediciones
- documentos
- document_chunks

Habilitar RLS no define todavía quién puede ver qué. Después necesitamos crear las políticas correspondientes.

Tabla compartida
dispositivos

id	tenant_id	nombre
1	1	Ambiente A01
2	1	Ambiente A02
3	2	Planta B01
4	2	Planta B02
5	3	Depósito C01
...	...	...

RLS habilitado

La base filtra las filas según el tenant actual



Política RLS

tenant_id

Filas autorizadas

Solo las filas del tenant de la sesión

id	tenant_id	nombre
1	1	Ambiente A01
2	1	Ambiente A02
...	...	...

Resultado para
tenant_id = 1

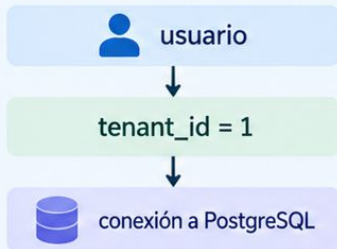


RLS habilita el mecanismo. Las políticas determinan qué filas resultan accesibles.

Contexto del tenant

¿Para qué organización estamos trabajando?

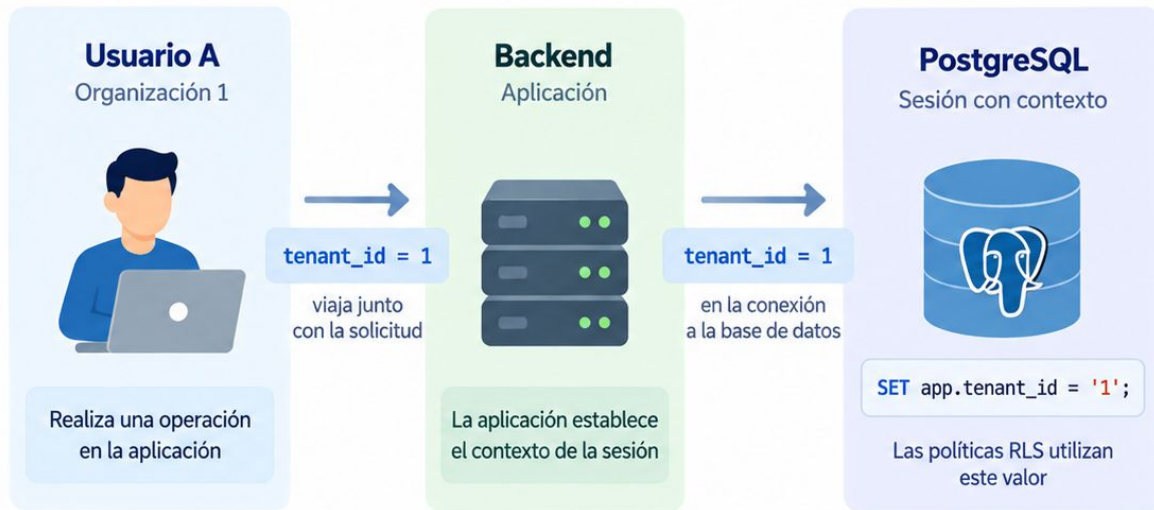
Cuando un usuario realiza una operación, la aplicación conoce la organización a la que pertenece.
Conceptualmente:



Podemos asociar ese valor a la sesión:

```
SET app.tenant_id = '1';
```

Las políticas RLS pueden utilizarlo posteriormente.



El contexto del usuario debe acompañar la operación hasta la capa de datos.

Política de lectura

Mostrar solamente las filas autorizadas

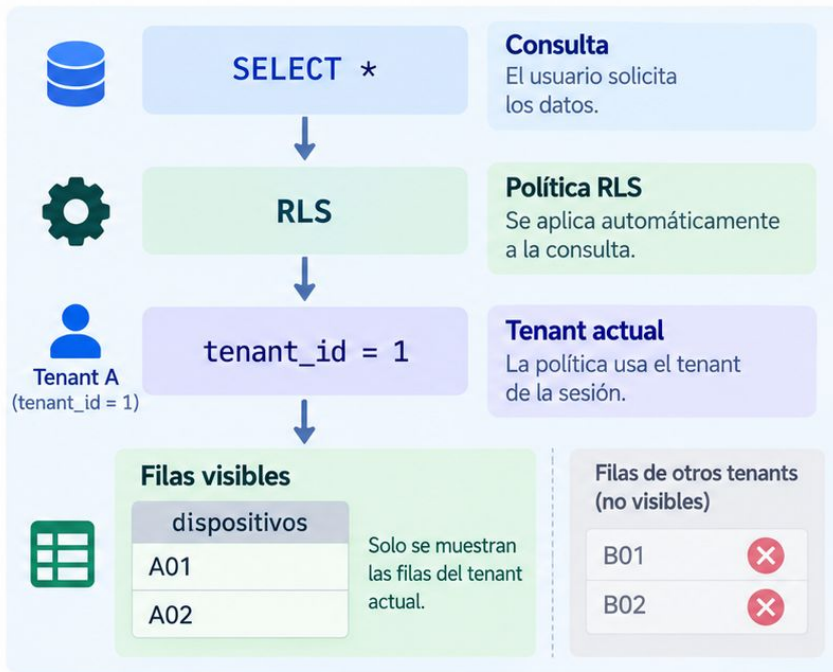
Política RLS (SQL)

```
CREATE POLICY tenant_select
ON dispositivos
FOR SELECT
USING (
    tenant_id =
    current_setting('app.tenant_id')::BIGINT
);
```

Consulta del usuario

```
SELECT *
FROM dispositivos;
```

La política agrega la restricción de acceso de manera automática.



La consulta pregunta por los datos. La política decide qué filas puede observar la sesión.

Leer no es el único riesgo

También debemos controlar qué filas pueden crearse o modificarse

Supongamos que el tenant A intenta insertar:

```
INSERT INTO dispositivos  
(tenant_id, nombre)  
VALUES (2, 'Planta B01');
```

Una política puede verificar que el registro nuevo corresponda al tenant de la sesión.

Conceptualmente:

```
WITH CHECK (  
  tenant_id =  
  current_setting('app.tenant_id')::BIGINT  
)
```

El mismo principio puede aplicarse a modificaciones (UPDATE).



Sesión actual
Tenant A

app.tenant_id = 1



Inserción permitida

El registro corresponde al tenant actual

```
INSERT INTO dispositivos  
(tenant_id, nombre)  
VALUES (1, 'Ambiente A03');
```



dispositivos

id	tenant_id	nombre
101	1	Ambiente A03



Inserción rechazada

El registro no corresponde al tenant actual

```
INSERT INTO dispositivos  
(tenant_id, nombre)  
VALUES (2, 'Planta B01');
```



Error de política RLS

No se puede insertar una fila para otro tenant.



El aislamiento debe proteger tanto la información que vemos como la que podemos modificar.

Dos niveles de autorización

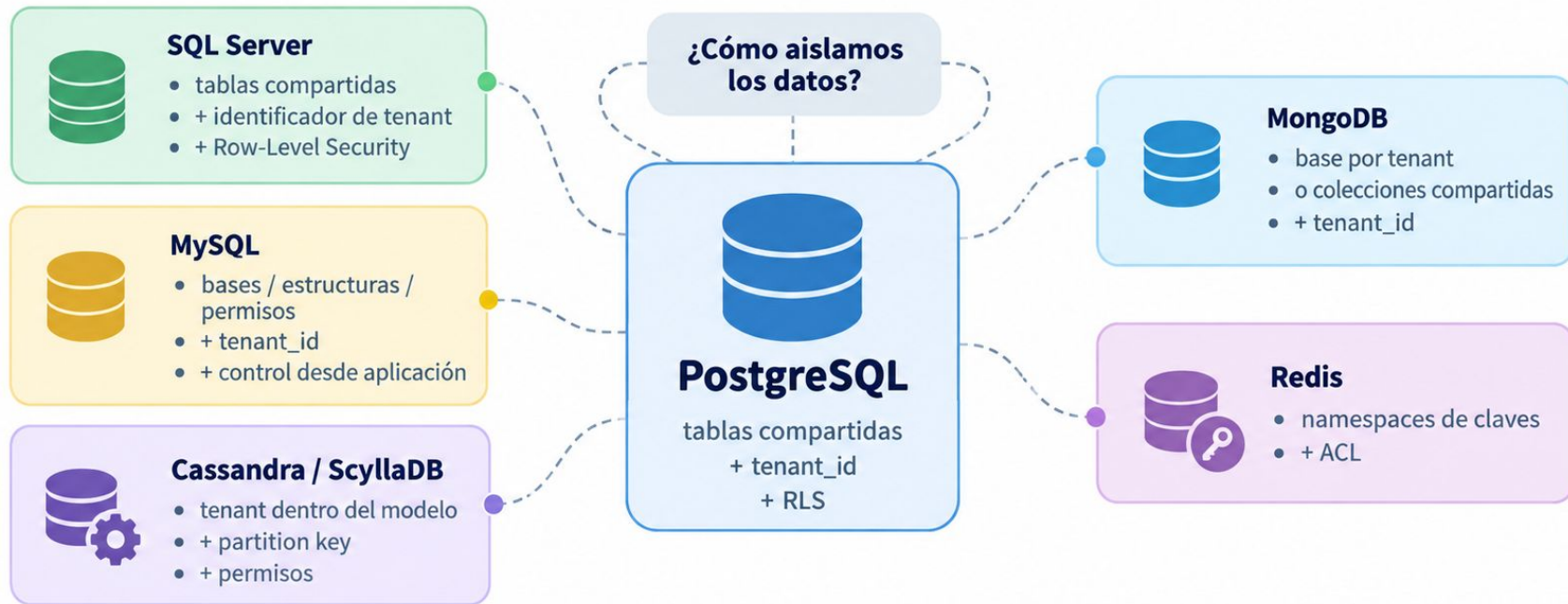
Qué operación y sobre qué datos



Los permisos controlan operaciones. RLS restringe los datos sobre los cuales esas operaciones pueden actuar.

Otros motores

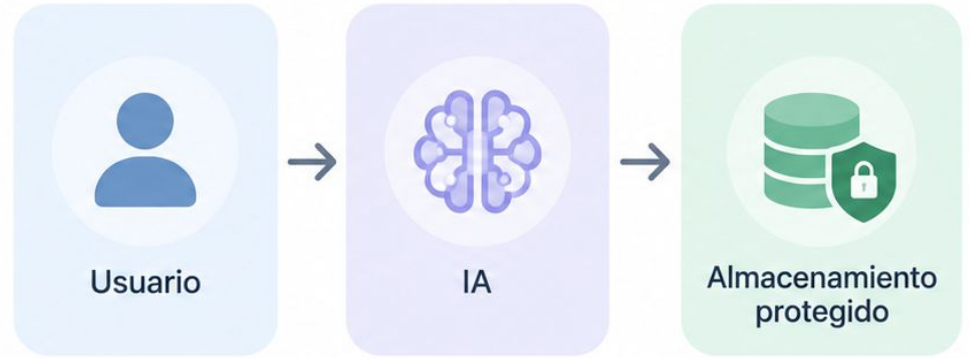
Mismo problema, mecanismos diferentes



Multi-tenant es un problema de arquitectura. Cada tecnología ofrece mecanismos diferentes para implementar el aislamiento.

3

RLS, RAG y Text-to-SQL



Una nueva forma de consultar

Pero las reglas de seguridad siguen siendo las mismas

En la Clase 6 incorporamos:



Text-to-SQL

Permite generar consultas en lenguaje natural.

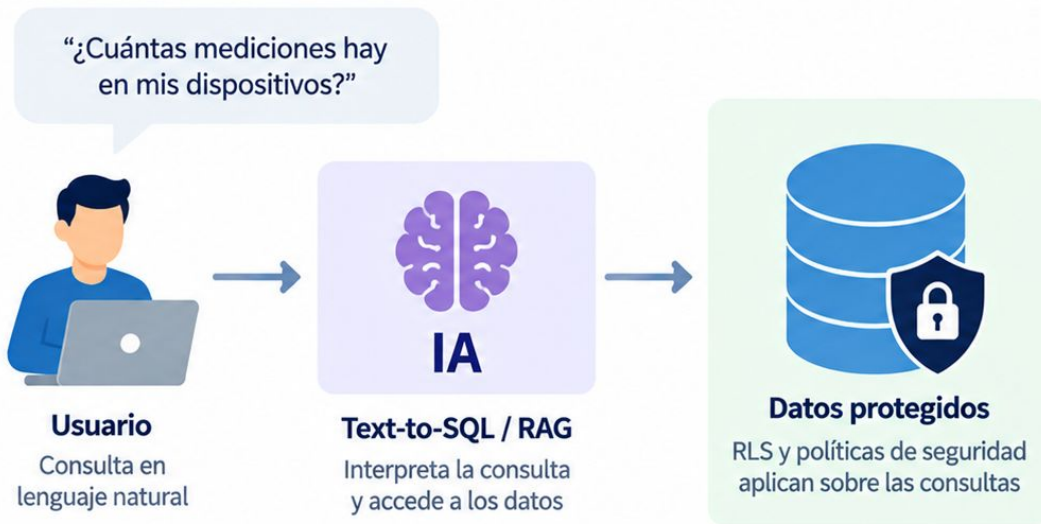


RAG

Recupera información relevante desde nuestros datos.

Ambos mecanismos terminan consultando información almacenada.

Ahora debemos preguntarnos:
¿qué información pueden utilizar para construir una respuesta?



Una IA no debería disponer de más información que el usuario para el cual está trabajando.

Text-to-SQL sin aislamiento

La consulta puede ser correcta y aun así ser peligrosa

Usuario pregunta:



“¿Cuántos dispositivos existen?”

El modelo genera:

```
SELECT COUNT(*)  
FROM dispositivos;
```

Si no existe aislamiento, podría contar dispositivos de todas las organizaciones.



Usuario

Consulta en
lenguaje natural



LLM / Text-to-SQL

Interpreta la consulta
y genera SQL



SQL

```
SELECT COUNT(*)  
FROM dispositivos;
```

Consulta generada

La consulta parece
correcta



**Consulta correcta,
acceso riesgoso**



Resultado sin aislamiento

Cuenta dispositivos de
todas las organizaciones

Organización	Dispositivos
Organización A	1,250
Organización B	980
Organización C	670
COUNT = todas	2,900

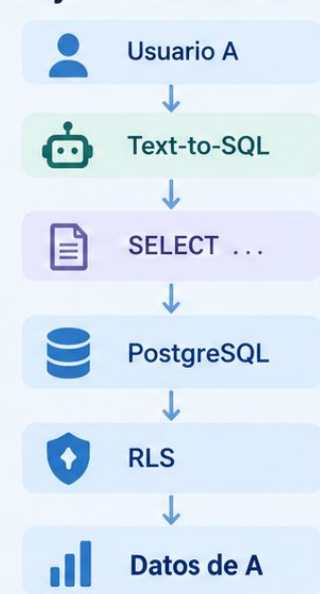


La seguridad no debería depender de que el LLM recuerde agregar `tenant\_id`.

La base mantiene el aislamiento

La consulta generada trabaja solamente con filas autorizadas

Flujo de la consulta



¿Qué sucede?



Consulta generada (parece general)

```
1 SELECT MAX(valor)
2 FROM mediciones
3 WHERE variable = 'temperatura';
```



La consulta no incluye un filtro por tenant, parece general.

Datos en la base (varios tenants)

id	tenant_id	variable	valor	fecha
101	A	temperatura	22.4	2024-05-01
102	A	temperatura	23.1	2024-05-02
201	B	temperatura	30.2	2024-05-01
202	B	temperatura	28.7	2024-05-02
...	...	...	...	...

Hay mediciones de otros tenants, pero RLS las mantiene fuera.

Resultado para Usuario A

variable	valor
temperatura	23.1



Solo participan las mediciones del tenant A.



RLS limita automáticamente qué mediciones participan.



La autorización debe aplicarse aunque la consulta haya sido generada automáticamente.

La política sigue aplicándose

Aunque la pregunta solicite información ajena

Usuario A pregunta:



“Mostrame las mediciones de la Organización B.”

La consulta puede intentar:

```
1 SELECT *
2 FROM mediciones
3 WHERE organization_id = 2;
```

RLS correspondiente al tenant A evita que esas filas sean visibles.

¿Qué sucede?



El modelo puede intentar una consulta. La base continúa aplicando sus reglas de acceso.

RAG también debe aislarse

La búsqueda vectorial no debería cruzar organizaciones

Los chunks de documentos se almacenan en una tabla compartida entre tenants.

document_chunks

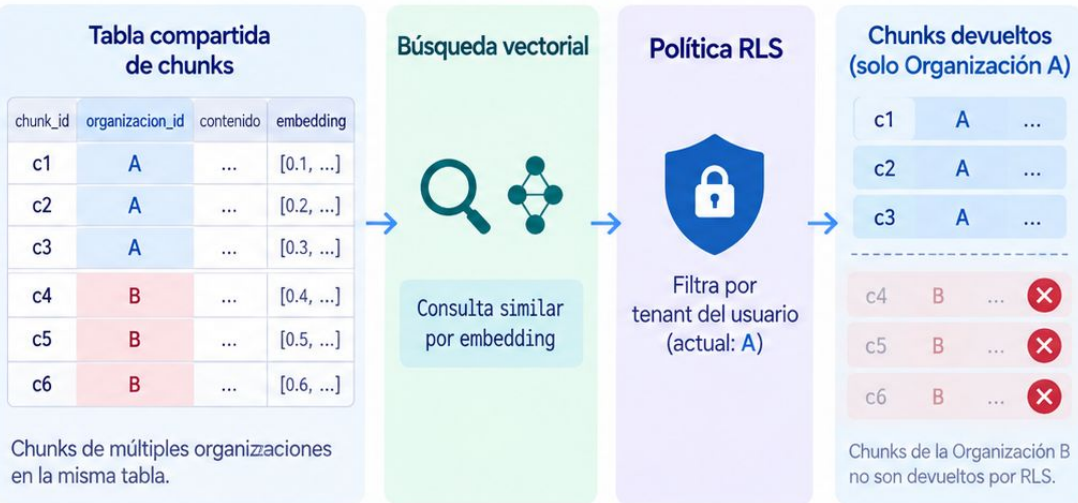
chunk_id
organizacion_id
contenido
embedding

Consulta:

```
SELECT contenido  
FROM document_chunks  
ORDER BY embedding <=> :query_embedding  
LIMIT 5;
```

RLS restringe qué chunks pueden recuperarse.

¿Cómo funciona la búsqueda vectorial con aislamiento?



El contenido que el usuario no puede consultar tampoco debería formar parte del contexto enviado al LLM.

Filtrar antes de generar

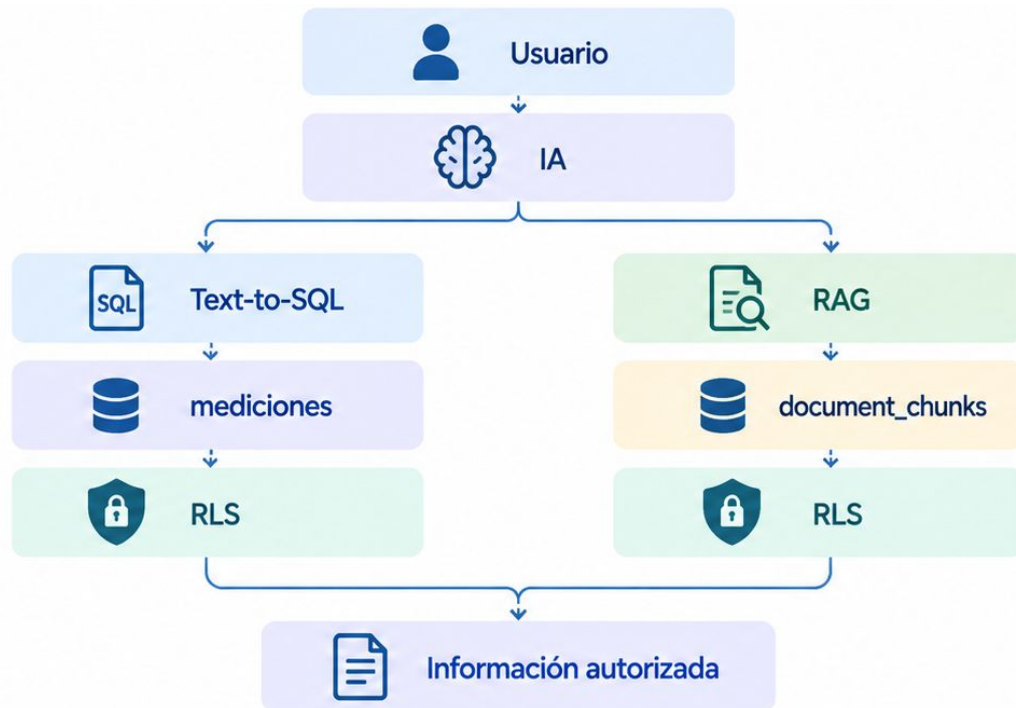
La seguridad ocurre antes del LLM, no después de generar la respuesta



Primero autorizamos la recuperación. Después generamos la respuesta.

Arquitectura integrada

Datos estructurados y documentos protegidos



Text-to-SQL y RAG cambian cómo consultamos. No cambian quién tiene permiso para acceder a la información.

4

Transacciones, Concurrencia y Bloqueos



ACID

Ahora desde la administración y la concurrencia



Atomicidad

La transacción se completa por completo o no se aplica.



Consistencia

Mantiene la integridad de los datos.



Aislamiento

Las transacciones no se afectan entre sí.



Durabilidad

Los cambios confirmados se mantienen ante fallos.

Ya conocimos las transacciones en Clase 3.

Ahora nos interesa especialmente:

aislamiento

+

concurrencia

Transacción 1



```
UPDATE cuenta  
SET saldo = saldo - 100  
WHERE id = 1;
```

Transacción 2



```
UPDATE cuenta  
SET saldo = saldo + 100  
WHERE id = 1;
```



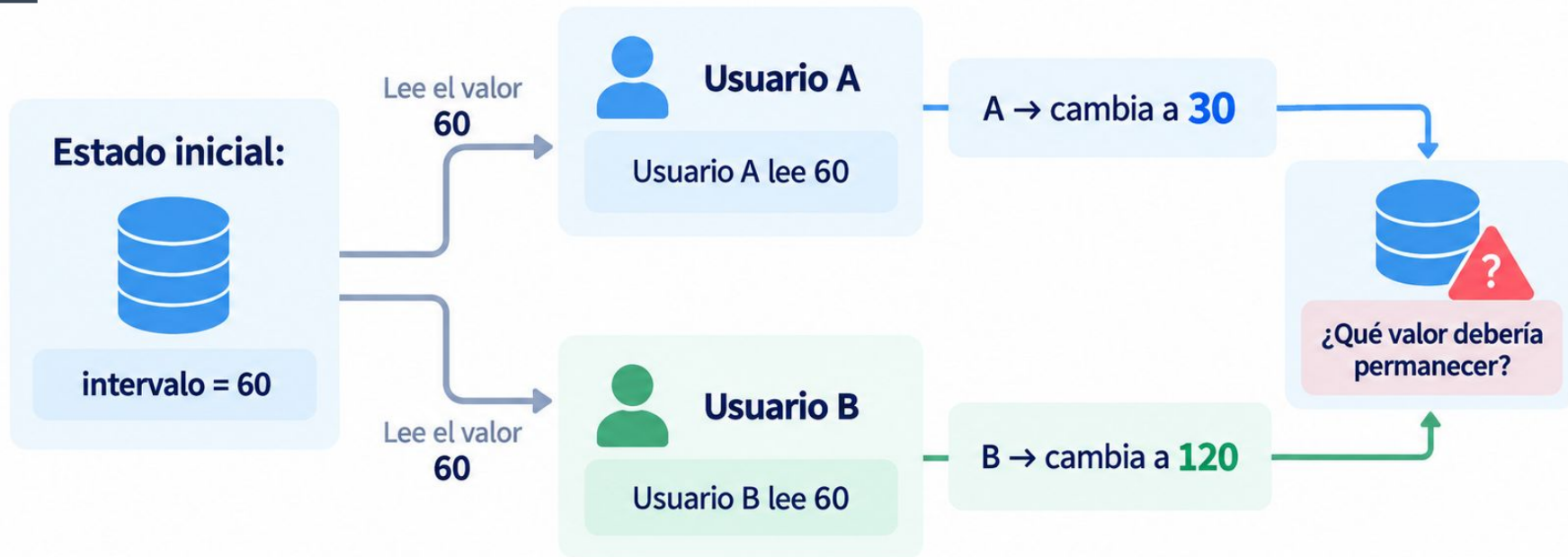
Mismo registro
accedido en forma simultánea



Una transacción no ocurre necesariamente sola dentro del sistema.

Operaciones simultáneas

¿Qué ocurre cuando ambos usuarios modifican a la vez?



La concurrencia aparece cuando varias operaciones trabajan simultáneamente sobre información relacionada.

Aislamiento

Las operaciones no deberían interferir de manera impredecible

El aislamiento controla cuánto puede observar una transacción sobre los cambios realizados por otras.

Podemos encontrar niveles como:



READ COMMITTED



REPEATABLE READ



SERIALIZABLE



**Más
concurrency**

Más transacciones
pueden trabajar
en paralelo



**Más
aislamiento**

Menos interferencias
entre transacciones



READ COMMITTED

Menor aislamiento,
mayor concurrency

REPEATABLE READ

Equilibrio entre
aislamiento y concurrency

SERIALIZABLE

Mayor aislamiento,
menor concurrency



Mayor aislamiento puede evitar determinados conflictos, pero también limitar la concurrency.

Múltiples versiones

Lecturas y escrituras pueden convivir mejor

PostgreSQL utiliza control de concurrencia mediante múltiples versiones.

Conceptualmente:

1 Transacción A observa una versión

2 Transacción B modifica otra versión

Esto reduce la necesidad de bloquear todas las lecturas mientras existen escrituras.



Lee la versión anterior sin bloquear toda la tabla.

Registro en PostgreSQL
(múltiples versiones)

id	valor	xmin	xmax
1	"ABC"	100	—
Versión 1 (anterior)			
1	"XYZ"	200	—
Versión 2 (nueva)			



Escribe una nueva versión del registro (sin bloquear las lecturas existentes).



Lecturas y escrituras pueden coexistir

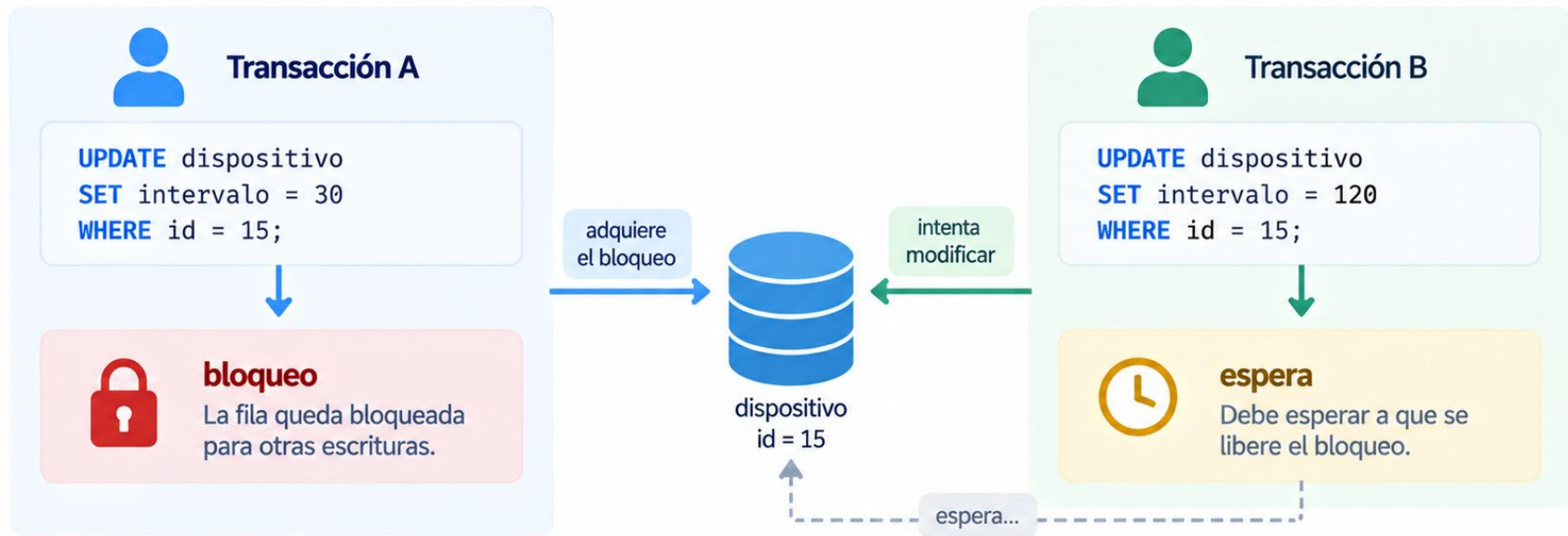
Cada transacción ve la versión que le corresponde.



Controlar concurrencia no significa bloquear cada operación.

Bloqueos

A veces una operación debe esperar



Los bloqueos ayudan a preservar consistencia cuando varias operaciones compiten por los mismos datos.

Deadlock

Dos transacciones pueden bloquearse mutuamente

A bloquea fila 1

B bloquea fila 2

A espera fila 2

B espera fila 1

El motor debe detectar el conflicto y abortar alguna de las operaciones.



La concurrencia puede generar situaciones que la aplicación debe estar preparada para manejar.

Observar actividad y bloqueos

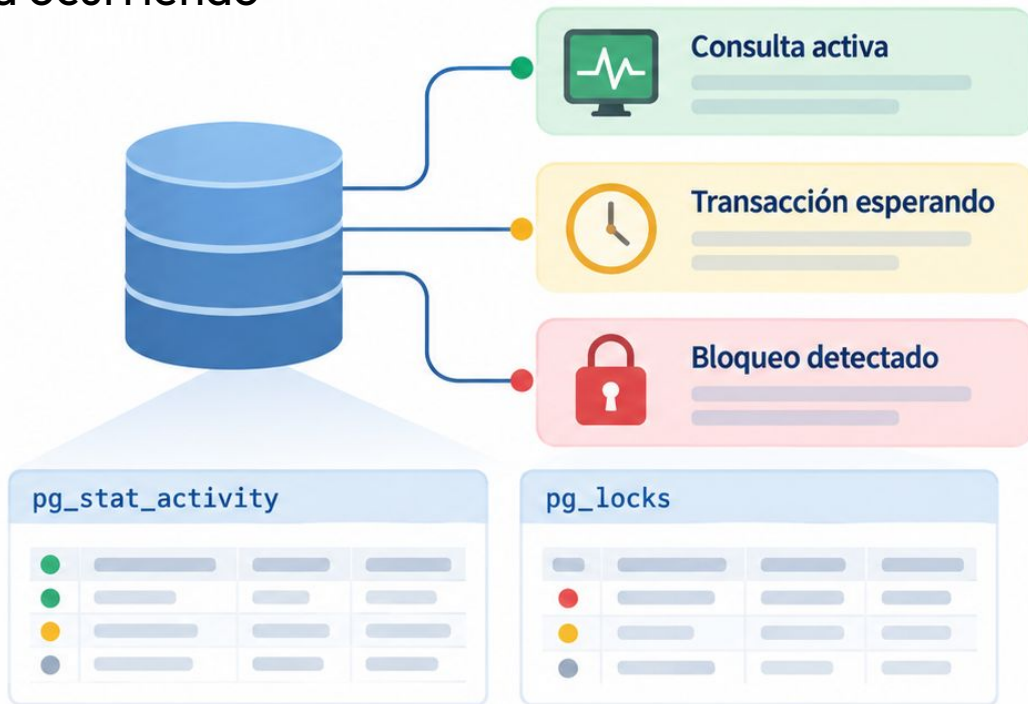
El motor puede mostrarnos qué está ocurriendo

Podemos consultar vistas como:

```
SELECT *  
FROM pg_stat_activity;
```

y analizar información relacionada
con bloqueos mediante:

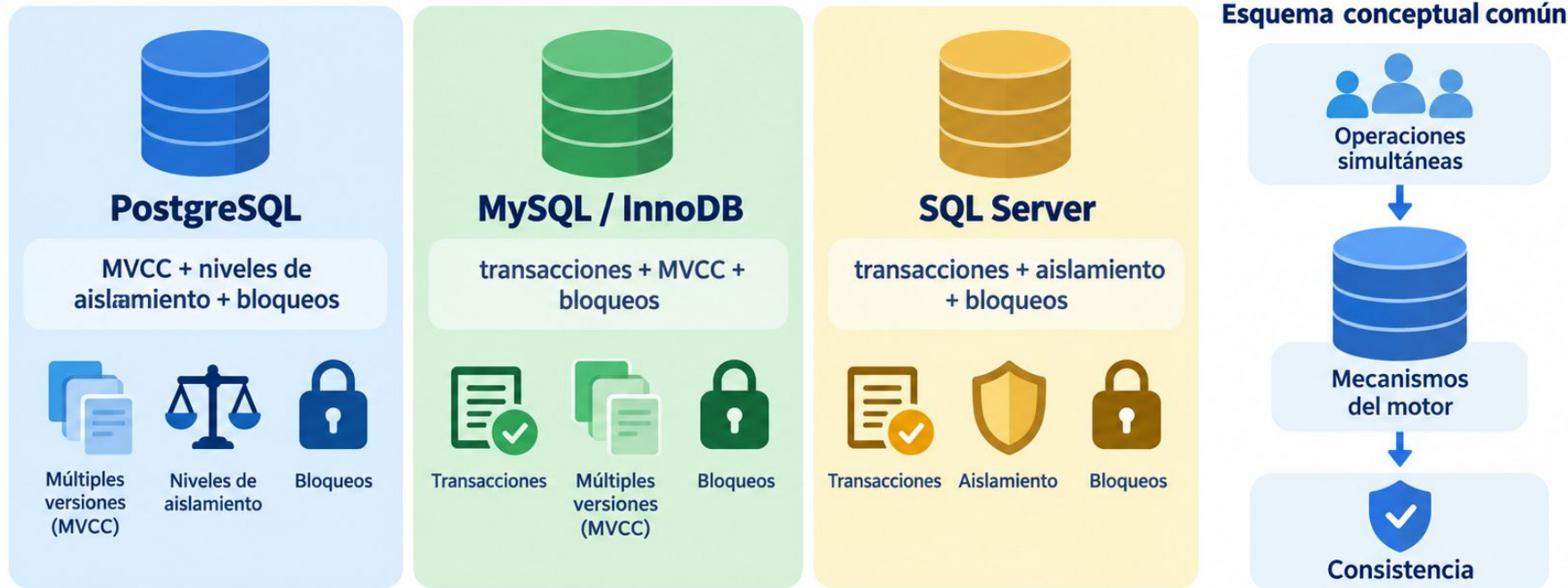
pg_locks



Administrar una base también implica poder observar operaciones que están esperando o compitiendo.

Otros motores relacionales

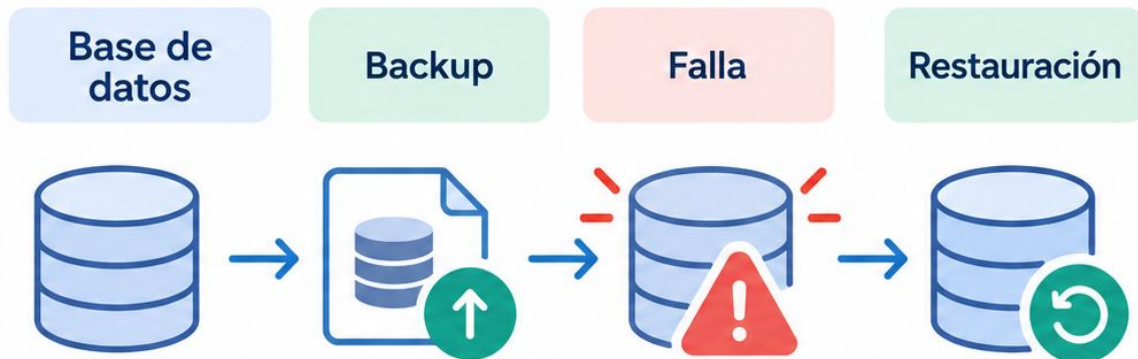
La concurrencia no es exclusiva de PostgreSQL



Las implementaciones pueden variar, pero el problema es el mismo: preservar consistencia frente a operaciones simultáneas.

5

Backup, Recuperación y Operación

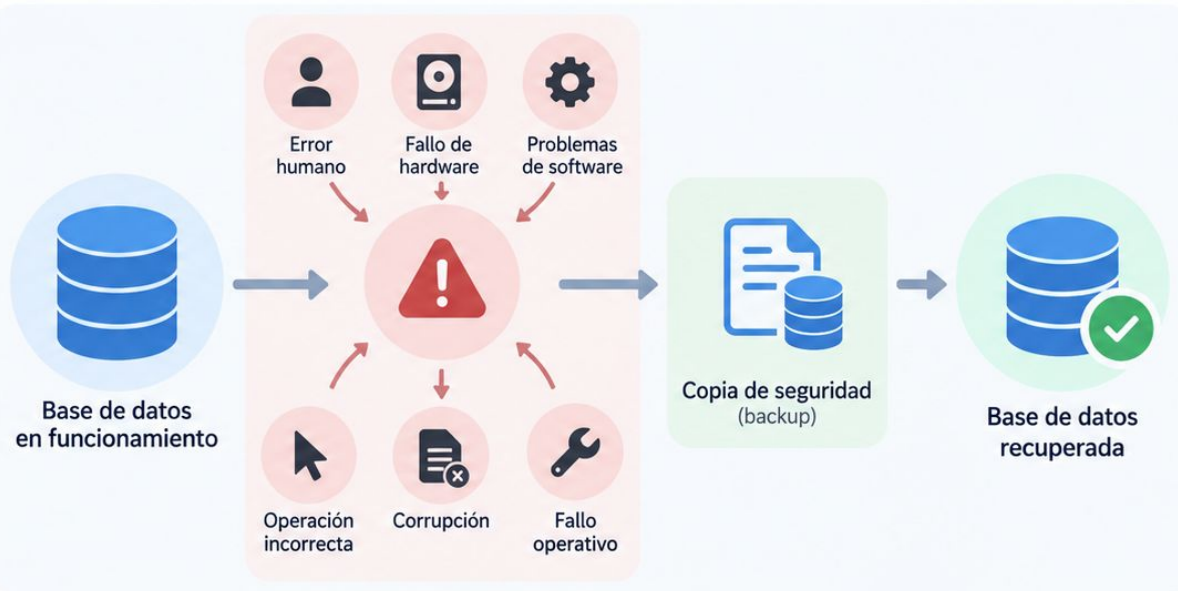


Tener datos no alcanza

¿Podemos recuperarlos?

Podemos perder información por:

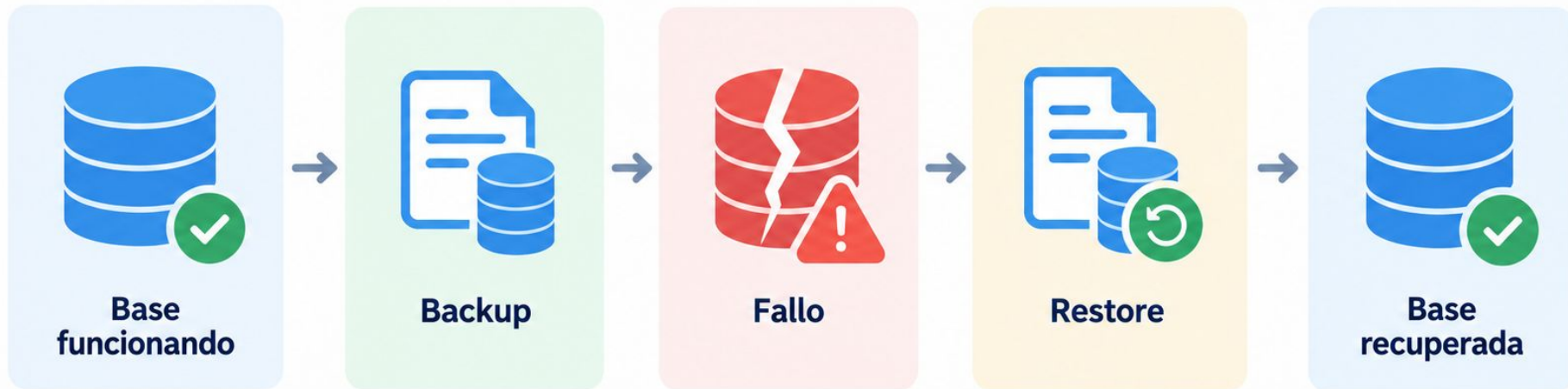
- errores humanos;
- fallos de hardware;
- problemas de software;
- operaciones incorrectas;
- corrupción;
- fallos operativos.



Una base es **recuperable** cuando tenemos un mecanismo para reconstruir su información después de un fallo.

Copiar y recuperar

Dos partes de la misma estrategia



Un backup que nunca fue probado mediante restauración todavía deja una pregunta sin responder.

Una copia lógica

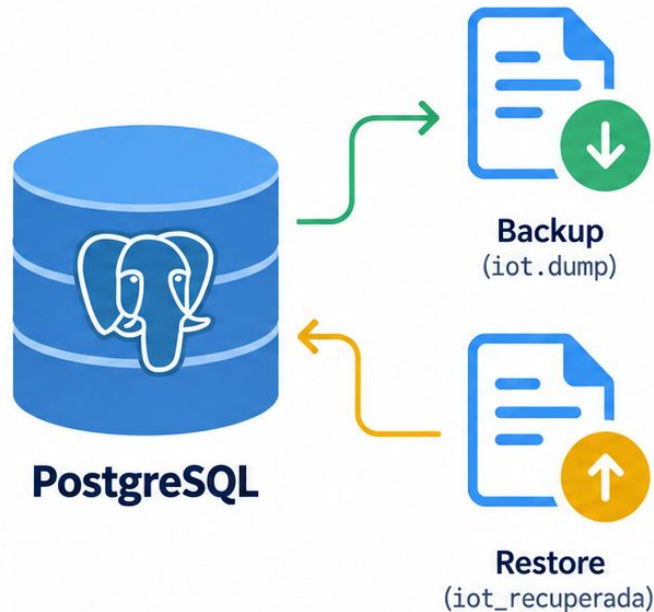
Herramientas que ya podemos utilizar

Backup:

```
$ pg_dump -Fc iot > iot.dump
```

Restore:

```
$ pg_restore -d iot_recuperada iot.dump
```



La estrategia concreta depende del tamaño, tiempo de recuperación y características del sistema.

WAL

PostgreSQL registra modificaciones

El Write-Ahead Log registra cambios antes de consolidarlos definitivamente.

Participa en mecanismos de:

- durabilidad;
- recuperación;
- replicación.



Registrar cambios permite reconstruir y propagar el estado de una base.

Dos objetivos diferentes

Disponibilidad y recuperación no son lo mismo

```
DELETE FROM mediciones;
```

Si el cambio se replica:

Primaria



DELETE

Réplica



DELETE



La segunda copia también puede recibir el error.



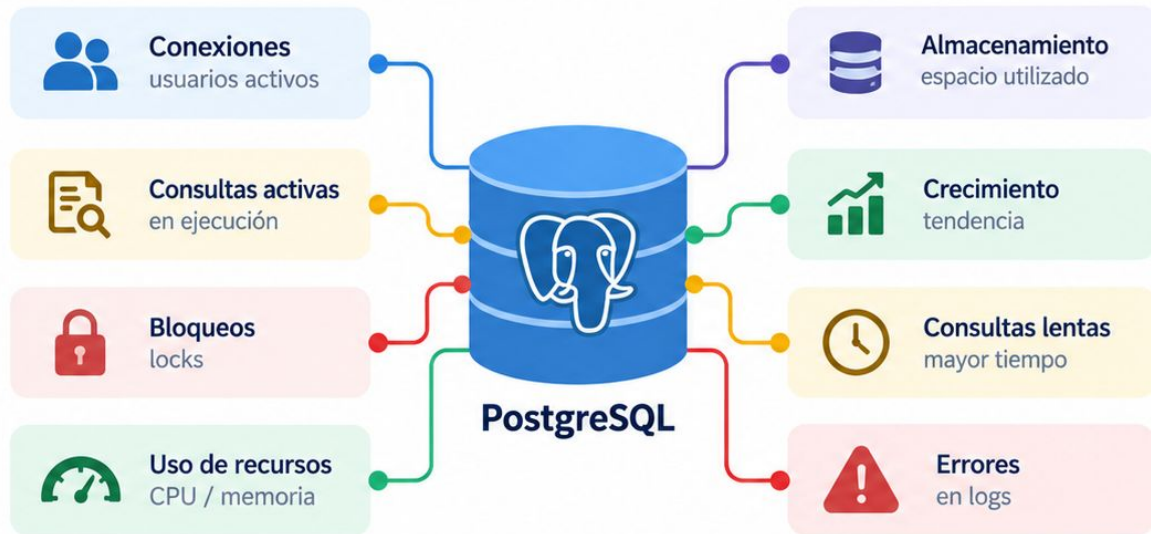
Replicar datos no elimina la necesidad de realizar backups.

¿Qué está ocurriendo?

Una base necesita observabilidad

Podemos observar:

- conexiones;
- consultas activas;
- bloqueos;
- utilización de recursos;
- almacenamiento;
- crecimiento;
- consultas lentas;
- errores.



Lo que no observamos resulta difícil de diagnosticar y optimizar.

Volvemos a conectar conceptos

Administración también significa detectar degradaciones



Consulta lenta

Mayor tiempo de ejecución



pg_stat_statements

Identifica las consultas más costosas



EXPLAIN ANALYZE

Analiza el plan de ejecución



consulta / índice / diseño

Ajusta y optimiza

También existen operaciones de mantenimiento como:



VACUUM

Recupera espacio y limpia registros muertos



ANALYZE

Actualiza las estadísticas del planificador



autovacuum

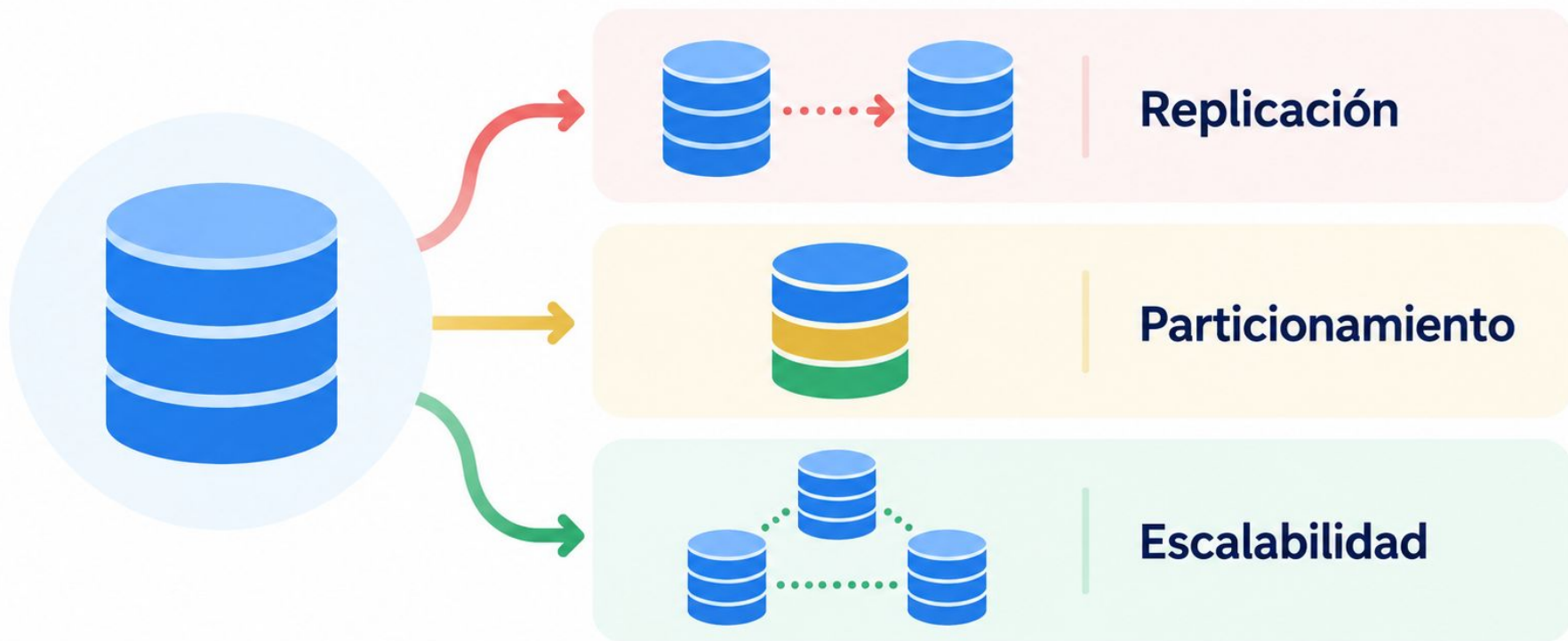
Proceso automático que mantiene la base de datos en buen estado



Optimización y administración forman parte del mismo ciclo operativo.

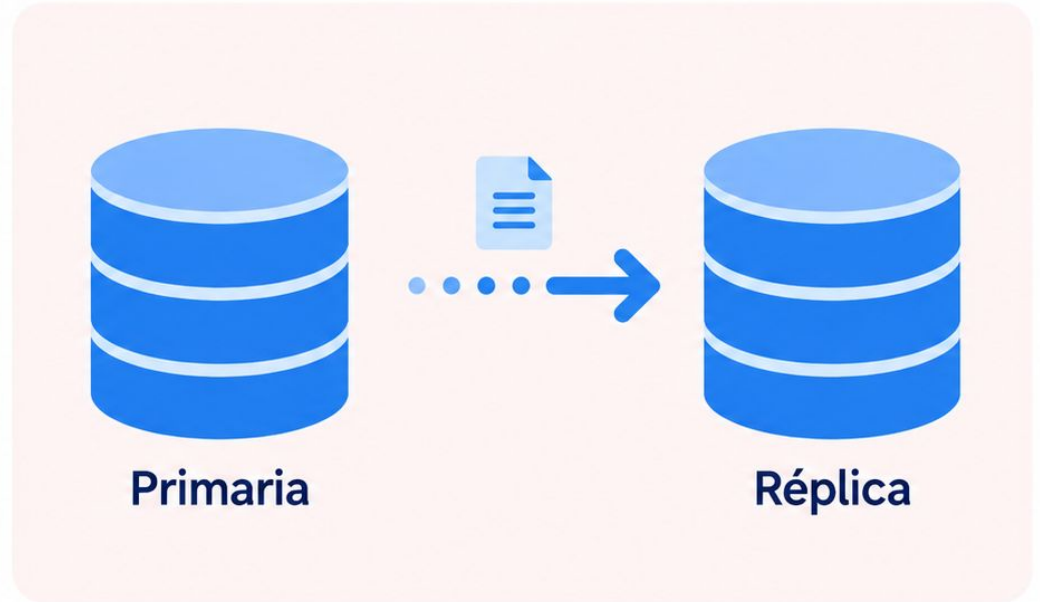
6

Replicación, Particionamiento y Escalabilidad



Mantener más de una copia

Los datos pueden existir en diferentes servidores



Replicar permite mantener copias operativas de la información.

¿Y si falla el servidor principal?

Una réplica puede formar parte de la estrategia de continuidad

Funcionamiento:



Fallo:



Alta disponibilidad busca reducir cuánto tiempo permanece indisponible el servicio.

Aplica a muchas tecnologías

No todas organizan las copias de la misma manera



Replicar es un concepto general. La arquitectura concreta depende del motor.

Una tabla puede crecer demasiado

Organizarla en partes puede facilitar su administración

Supongamos:

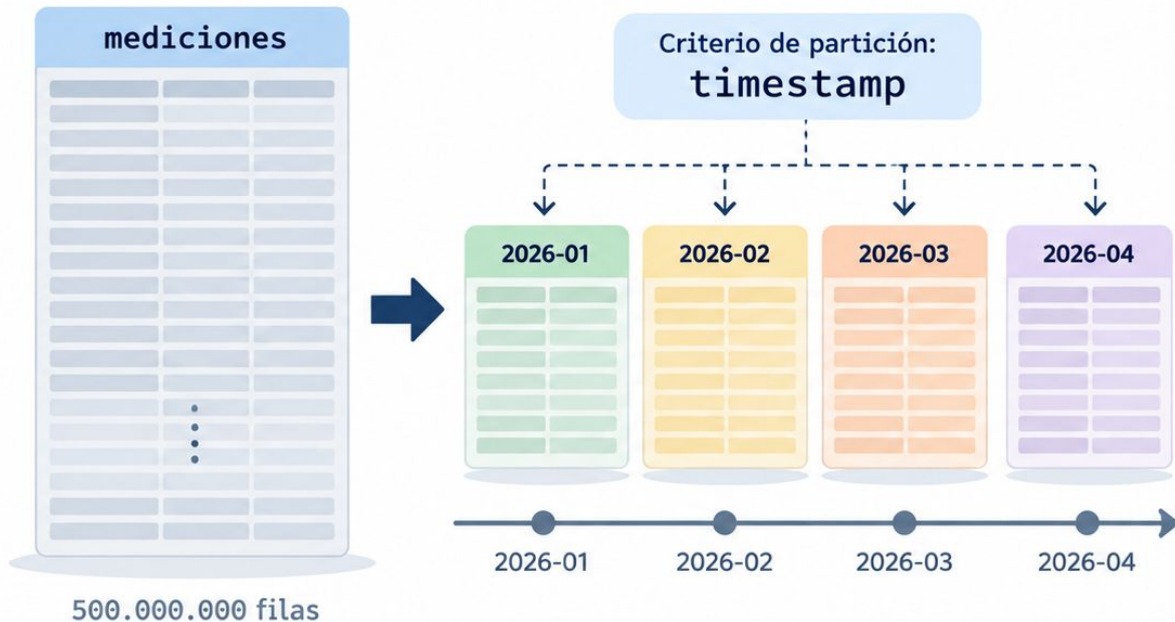
mediciones

500.000.000 filas

La mayoría de las consultas utilizan:

timestamp

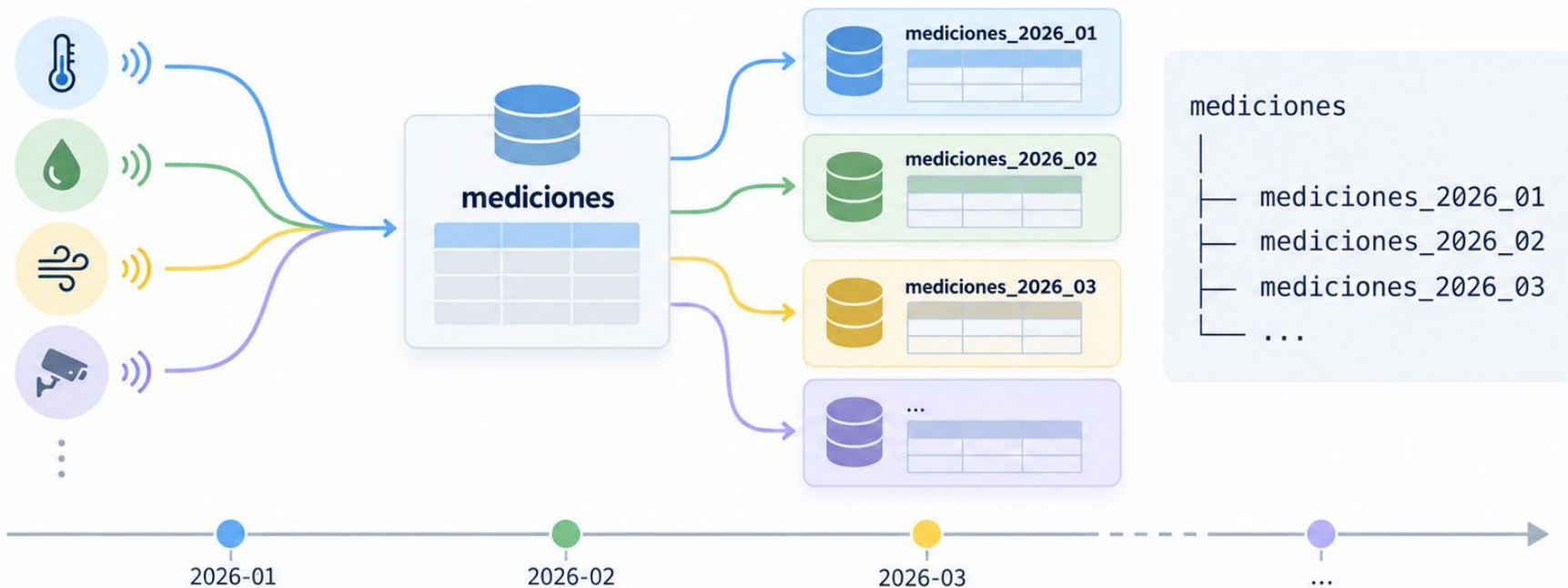
Podemos analizar
particionamiento por períodos.



Particionar significa dividir lógicamente un conjunto grande de datos según algún criterio.

Un caso natural para IoT

El histórico crece siguiendo el tiempo



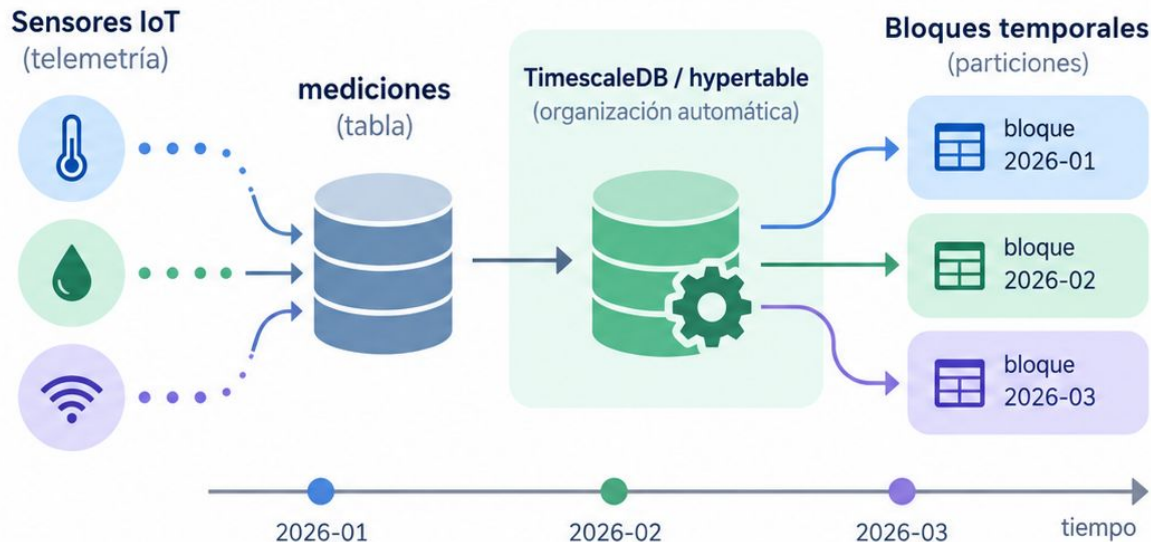
El criterio de particionamiento debería relacionarse con cómo crecen y cómo se consultan los datos.

Un concepto que ya utilizamos

Ahora podemos comprender mejor qué problema resuelve

En Clase 6 utilizamos una hypertable para organizar telemetría.

mediciones
↓
organización automática
↓
por bloques temporales



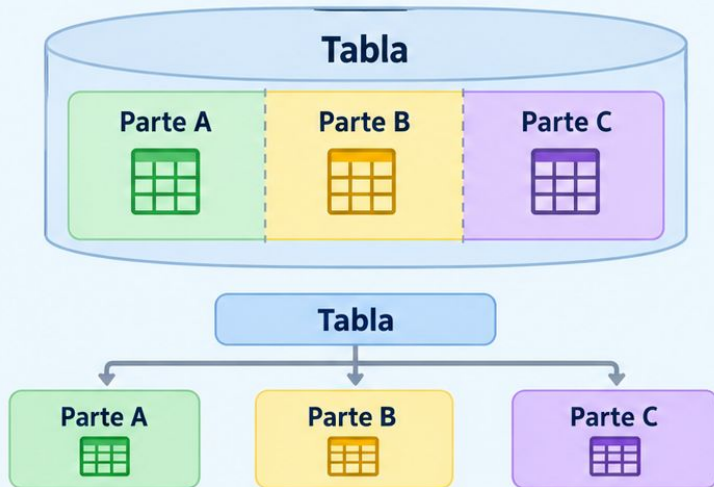
Las tecnologías especializadas pueden automatizar decisiones que también existen en el diseño general de una base.

Dos conceptos distintos

Dividir datos y repartirlos entre nodos no son exactamente lo mismo

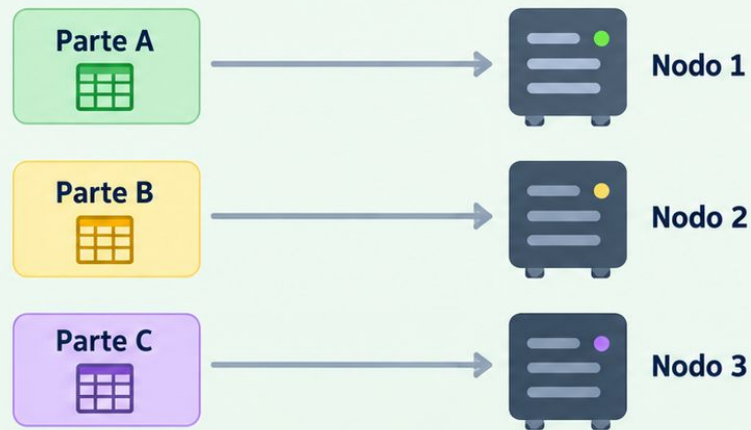
Particionamiento

Una misma tabla se divide en partes dentro de la misma base de datos.



Distribución

Las partes de la tabla se reparten entre diferentes nodos.



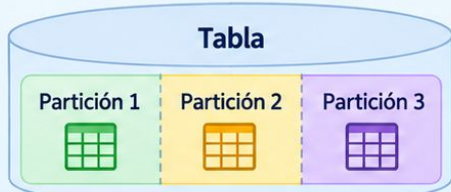
Podemos particionar sin distribuir. En una arquitectura distribuida ambas ideas pueden combinarse.

Estructuras diferentes

El criterio depende del modelo y de la arquitectura

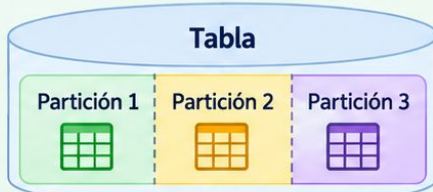
PostgreSQL

particiones de tablas



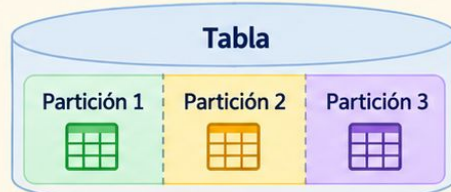
MySQL

partitioning



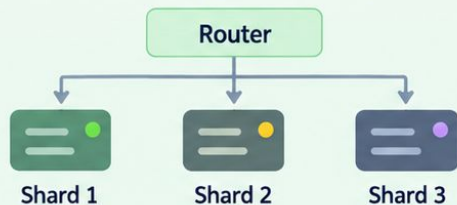
SQL Server

particiones de tablas



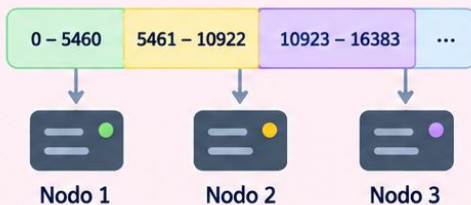
MongoDB

sharding + shard key



Redis Cluster

hash slots



Cassandra / ScyllaDB

partition key



En sistemas distribuidos, la forma de particionar también puede determinar dónde termina almacenado el dato.

Escalabilidad vertical

Aumentar la capacidad de una máquina

Servidor inicial



Servidor mayor



Escalar verticalmente aumenta los recursos disponibles en un nodo.

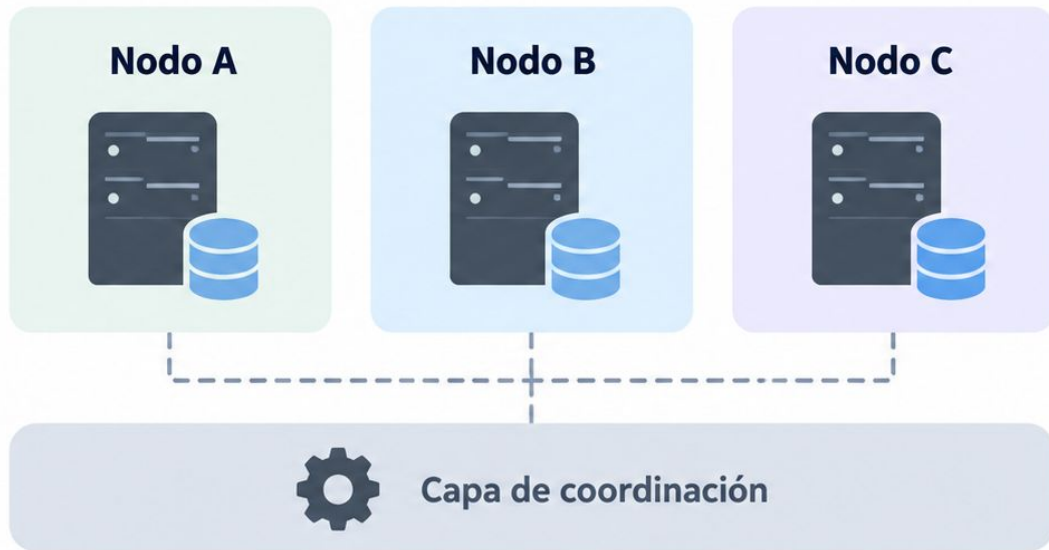
Escalabilidad horizontal

Agregar nodos a la arquitectura

Nodo A + Nodo B + Nodo C

Esto requiere decidir:

- qué datos se replican;
- qué datos se distribuyen;
- cómo se coordinan los nodos.



Agregar máquinas cambia la arquitectura y también introduce problemas de distribución.

Puntos de partida distintos

La arquitectura condiciona cómo puede crecer el sistema

PostgreSQL / MySQL / SQL Server



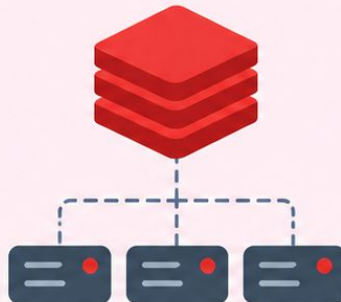
- escalado vertical
- + réplicas
- + particionamiento
- + arquitecturas adicionales

MongoDB



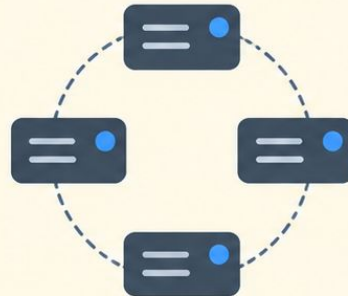
replicación + sharding

Redis



replicación + cluster

Cassandra / ScyllaDB



distribución horizontal
como parte del modelo



La escalabilidad no depende solamente del motor, sino también de cómo diseñamos toda la solución.

No vamos a distribuir todo todavía

La siguiente clase amplía el problema

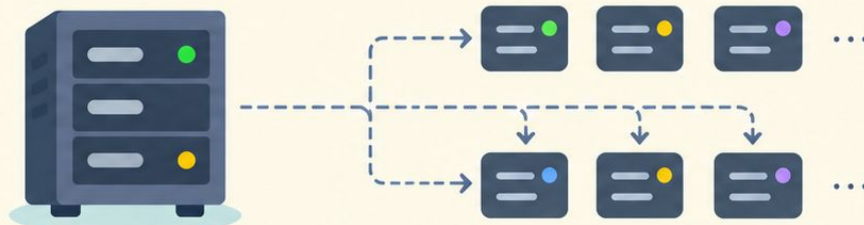
Hoy introducimos:

- réplica;
- partición;
- escalabilidad vertical;
- escalabilidad horizontal.



Todavía quedan preguntas:

- ¿qué ocurre con cientos de millones de registros?;
- ¿cómo distribuimos procesamiento?;
- ¿cómo procesamos flujos continuos?;
- ¿cómo separamos operación y analítica?



La escalabilidad de una base será el punto de partida para pensar arquitecturas de datos a mayor escala.

7

Práctica Integradora



Evolucionar el proyecto existente

Vamos a proteger el sistema que ya construimos

El proyecto ya contiene:

- organizaciones
- ubicaciones
- dispositivos
- mediciones
- documentos
- document_chunks



Vamos a incorporar:

tenant_id / organizacion_id

+

roles

+

RLS

+

IA con acceso controlado



La seguridad se incorpora sobre una arquitectura que ya existe.

Dos tenants

Cargar información perteneciente a organizaciones diferentes



Después:

- establecer tenant actual;
- ejecutar consultas;
- comprobar qué filas resultan visibles.



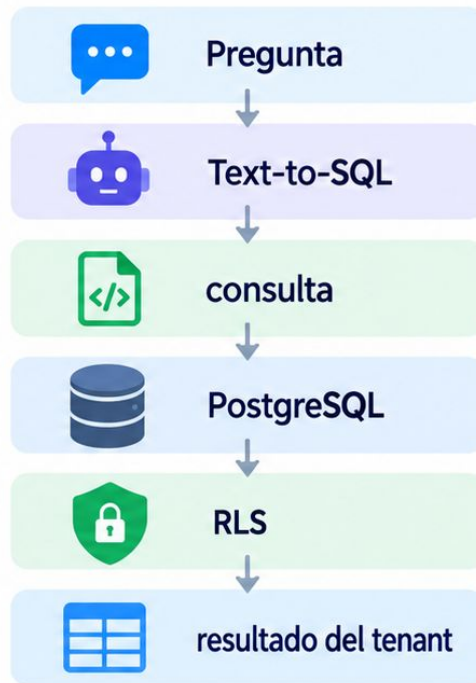
Primero vamos a comprobar el aislamiento directamente sobre la base.

Consultas estructuradas

La IA utiliza el mismo aislamiento que la aplicación

Ejemplo:

“¿Cuál fue la temperatura máxima registrada hoy?”



Después intentar:

“Mostrame las mediciones de la otra organización.”



Acceso denegado

No puedes acceder a datos de otra organización.



La consulta generada por IA no debe poder superar los permisos del usuario.

Consultar documentación

La recuperación también debe respetar el tenant

Los chunks contienen:



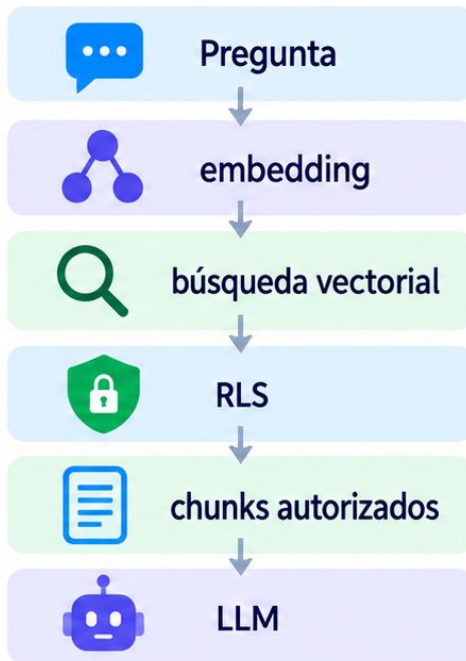
organizacion_id



contenido



embedding



Probar:

“Haceme una pregunta sobre un documento de otro tenant.”



Sin acceso



La información no autorizada debe quedar fuera del contexto antes de generar la respuesta.

Política única de accesos

Aplicación, SQL e IA utilizan la misma arquitectura de seguridad



La seguridad no pertenece a la interfaz ni al modelo de IA. Forma parte de la arquitectura de datos.



Para ir cerrando..

Errores frecuentes

Problemas típicos al administrar y proteger una base de datos



Seguridad y aislamiento

- Conectar la aplicación utilizando un usuario administrador.
- Otorgar más permisos de los necesarios.
- Depender solamente del código de la aplicación para aislar tenants.
- Suponer que agregar `tenant\_id` garantiza por sí solo el aislamiento.
- Aplicar RLS solamente a algunas tablas que contienen datos del tenant.
- Permitir que RAG o Text-to-SQL accedan con permisos superiores a los del usuario.
- Ejecutar consultas generadas automáticamente sin controles.



Operación, respaldo y escalado

- Ignorar operaciones concurrentes y posibles bloqueos.
- Confundir una réplica con un backup.
- Realizar backups sin comprobar que puedan restaurarse.
- Particionar sin analizar cómo crecen y se consultan los datos.
- Escalar agregando infraestructura sin identificar primero el cuello de botella.



Una arquitectura puede funcionar técnicamente y aun así ser insegura, difícil de recuperar o innecesariamente compleja.

Buenas prácticas

Administrar y proteger con criterio



Buenas prácticas

- Separar usuarios administrativos y usuarios de aplicación.
- Aplicar el principio de mínimo privilegio.
- Definir explícitamente a qué tenant pertenece cada dato.
- Elegir conscientemente la estrategia de aislamiento.
- Aplicar las políticas de seguridad lo más cerca posible de los datos.
- Mantener los mismos controles para aplicaciones, RAG y Text-to-SQL.
- Limitar qué puede ejecutar una consulta generada automáticamente.
- Diseñar transacciones considerando posibles operaciones concurrentes.
- Mantener y probar una estrategia de backup y restore.
- Monitorear conexiones, bloqueos, crecimiento y consultas lentas.
- Particionar solamente cuando exista un problema o patrón que lo justifique.
- Medir antes de decidir cómo escalar.



Identidad



Permisos



Aislamiento



Monitoreo



Recuperación



Escalabilidad



Seguridad, recuperación y escalabilidad deben diseñarse antes de que aparezca el problema.

Antes de poner en producción

Preguntas para revisar la arquitectura



Preguntas clave

- ¿Quién se conecta a la base?
- ¿Qué necesita hacer cada usuario o aplicación?
- ¿Estamos otorgando solamente los permisos necesarios?
- ¿Existen varios tenants?
- ¿Cómo se encuentran aislados?
- ¿Qué ocurre si una consulta olvida el filtro del tenant?
- ¿RAG y Text-to-SQL respetan los mismos permisos?
- ¿Qué ocurre si dos operaciones modifican el mismo dato?
- ¿Podemos recuperar la base después de un fallo o error humano?
- ¿Sabemos cuánto está creciendo?
- ¿Podemos detectar consultas lentas o bloqueos?
- ¿Necesitamos realmente replicar, particionar o escalar?



Operar una arquitectura significa pensar también qué puede fallar y cómo vamos a detectarlo o recuperarlo.

¿Qué observamos?

De almacenar datos a operar una arquitectura

Durante la clase:

- diferenciamos autenticación y autorización;
- utilizamos usuarios, roles y permisos;
- aplicamos mínimo privilegio;
- analizamos tres estrategias de aislamiento multi-tenant;
- incorporamos `tenant\_id`;
- utilizamos Row Level Security;
- extendimos el aislamiento a RAG y Text-to-SQL;
- analizamos concurrencia y bloqueos;
- distinguimos backup, recuperación y replicación;
- incorporamos monitoreo y mantenimiento;
- analizamos particionamiento;
- diferenciamos escalabilidad vertical y horizontal.



La arquitectura de datos no termina en el almacenamiento: también define cómo se protege, opera, recupera y hace crecer la información.

Resumen de la clase

Ideas principales

- 1 Los **roles y permisos** controlan qué operaciones puede realizar cada conexión.
- 2 El **mínimo privilegio** reduce accesos innecesarios.
- 3 Una aplicación **multi-tenant** necesita una estrategia explícita de aislamiento.
- 4 El aislamiento puede realizarse mediante **bases, esquemas o tablas compartidas**.
- 5 En tablas compartidas, `'tenant_id'` identifica la pertenencia de cada fila.
- 6 **RLS** permite aplicar políticas de acceso a nivel de fila.
- 7 **RAG y Text-to-SQL** deben respetar los mismos límites que el usuario.
- 8 La **concurrency** puede generar conflictos y bloqueos.
- 9 **Backup y replicación** resuelven problemas diferentes.
- 10 El **monitoreo** permite observar el comportamiento real de la base.
- 11 El **particionamiento** organiza grandes conjuntos de datos.
- 12 La **escalabilidad vertical y horizontal** representan estrategias diferentes de crecimiento.

Mapa conceptual

De la seguridad al crecimiento



Una arquitectura de datos debe ser útil, pero también segura, observable, recuperable y capaz de evolucionar.

Próxima clase

Arquitecturas de Datos y Grandes Volúmenes

Hasta ahora aprendimos a diseñar, almacenar, proteger y administrar los datos.

En la próxima clase vamos a analizar qué ocurre cuando la arquitectura comienza a encontrar límites por:

- volumen;
- velocidad;
- variedad;
- distribución;
- procesamiento analítico;
- flujos continuos de información.

Vamos a introducir:

- Big Data y las 5 Vs;
- sistemas distribuidos;
- nodos, clústeres y particiones;
- OLTP y procesamiento analítico;
- batch, microbatch y streaming;
- Data Warehouos;
- Data Lake;
- Data Lakehouse.



Hoy aprendimos a preparar una base para operar y crecer.

La próxima clase veremos cuándo el problema requiere pensar una arquitectura de datos a otra escala.



Enlace a la encuesta



Muchas gracias ¿Dudas?